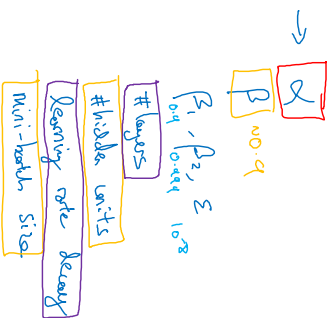
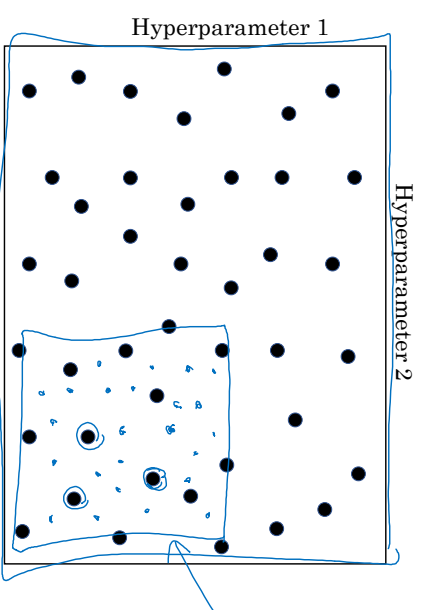


Hyperparameters



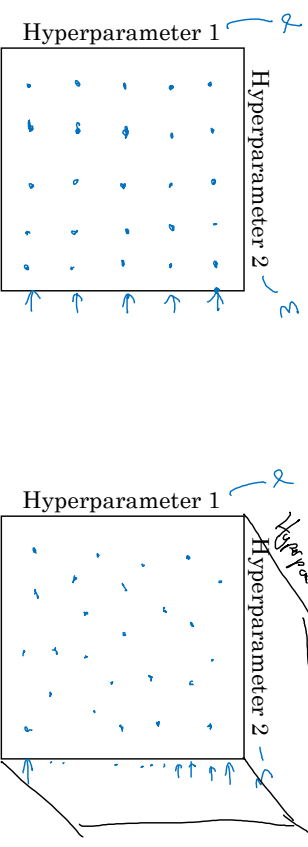
Andrew Ng

Coarse to fine



Andrew Ng

Try random values: Don't use a grid



Andrew Ng



deeplearning.ai

Hyperparameter tuning

Tuning process

Picking hyperparameters at random

$$\rightarrow \eta^{trial} = S_0, \dots, 100$$



$$\rightarrow \#layers \quad L: \quad 2, 3, 4$$

$$2, 3, 4$$

Andrew Ng

Hyperparameters for exponentially weighted averages

$$\beta = 0.9 \quad \dots \quad 0.999$$

$$10 \quad \dots \quad 1000$$

$$1-\beta = 0.1 \quad \dots \quad 0.001$$

$$\beta: 0.999 \rightarrow 0.9005 \quad \sim 10$$

$$\beta: 0.999 \rightarrow 0.9995 \quad \sim 1000$$

$$\frac{1}{\beta_K}$$

$$0.9 \quad \dots \quad 0.999$$

$$0.9 \quad \dots \quad 0.999$$

$$0.1 \quad \dots \quad 0.001$$

$$10^{-1} \quad \dots \quad 10^{-3}$$

$$\beta \in [-3, -1]$$

$$1-\beta = 10^r$$

$$\beta = 1-10^r$$

Andrew Ng



deeplearning.ai

Hyperparameter tuning

Using an appropriate scale to pick hyperparameters

Appropriate scale for hyperparameters

$$\alpha = 0.0001, \dots, 1$$

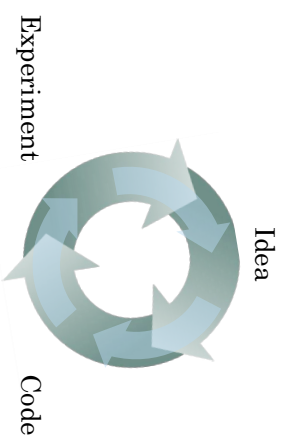
$$0.0001 \quad \dots \quad 1$$

$$10^{-4} \quad \dots \quad 10^0$$

$$10^{-4} \quad \dots \quad 10^0$$

Andrew Ng

Re-test hyperparameters occasionally



- NLP, Vision, Speech, Ads, logistics,
- Intuitions do get stale. Re-evaluate occasionally.

Andrew Ng



deeplearning.ai

Hyperparameters tuning

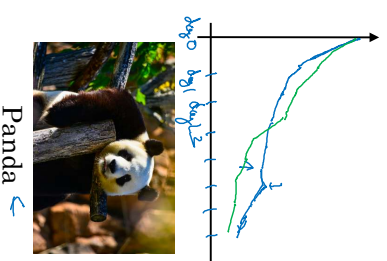
Hyperparameters tuning in practice: Pandas vs. Caviar



deeplearning.ai

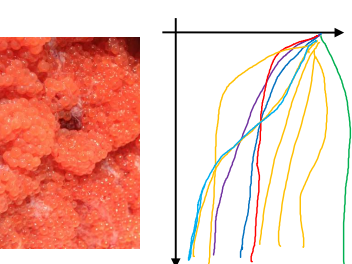
Batch Normalization Normalizing activations in a network

Babysitting one model



Panda ←

Training many models in parallel



Caviar ←

Andrew Ng

Implementing gradient descent

for $t = 1 \dots \text{num MiniBatches}$

Compute forward pass on $X^{\{t\}}$.

If each hidden layer, use RN to map $\underline{z}^{(l)}$ with $\underline{z}^{(l+1)}$.

Use backward pass to compute $\underline{d}^{(l)}$, $\underline{d}^{(l+1)}$, $\underline{d}^{(l+2)}$, $\underline{d}^{(l+3)}$.

Update parameters

$$\begin{aligned} W^{(l+1)} &:= W^{(l)} - \alpha \underline{d}^{(l+1)} \\ \beta^{(l+1)} &:= \beta^{(l)} - \alpha \underline{d}^{(l+2)} \\ \beta^{(l+3)} &:= \beta^{(l+2)} - \alpha \underline{d}^{(l+3)} \end{aligned}$$

Works w/ momentum, RMSprop, Adam.

Andrew Ng

Working with mini-batches

$$\tilde{X}^{\{t\}} \xrightarrow{W^{(1)}, b^{(1)}} \underline{z}^{(1)} \xrightarrow{\frac{\beta^{(1)} g^{(1)}}{0.01}} \underline{z}^{(2)} \rightarrow g^{(2)}(\underline{z}^{(2)}) \xrightarrow{W^{(2)}, b^{(2)}} \underline{z}^{(3)} \rightarrow \dots$$

$$\tilde{X}^{\{t\}} \xrightarrow{\beta^{(1)} g^{(1)}} \underline{z}^{(1)} \xrightarrow{\frac{\beta^{(1)} g^{(1)}}{0.01}} \underline{z}^{(2)} \rightarrow \dots$$

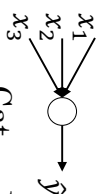
Parameters: $W^{(1)}, \beta^{(1)}, \beta^{(2)}, \beta^{(3)}$.

$$\begin{aligned} \underline{z}^{(1)} &= W^{(1)} \tilde{X}^{\{t\}} + b^{(1)} \\ \underline{z}^{(2)} &= W^{(2)} \underline{z}^{(1)} + b^{(2)} \\ \underline{z}^{(3)} &= W^{(3)} \underline{z}^{(2)} + b^{(3)} \end{aligned}$$

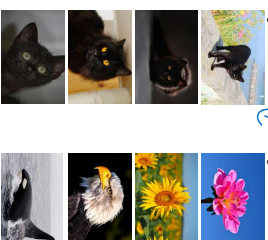
$$\begin{aligned} \underline{z}^{(1)} &= W^{(1)} \tilde{X}^{\{t\}} + b^{(1)} \\ \underline{z}^{(2)} &= W^{(2)} \underline{z}^{(1)} + b^{(2)} \\ \underline{z}^{(3)} &= W^{(3)} \underline{z}^{(2)} + b^{(3)} \end{aligned}$$

Andrew Ng

Learning on shifting input distribution

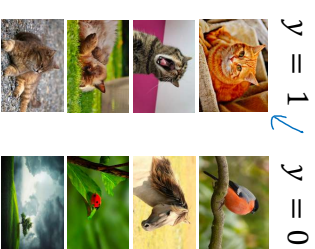


Cat $y = 1$
Non-Cat $y = 0$



"covariate shift"

$$X \rightarrow Y$$



Andrew Ng



deeplearning.ai

Why does Batch Norm work?

Batch Normalization

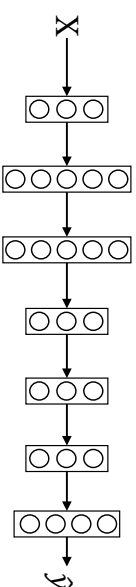
Batch Norm as regularization

- Each mini-batch is scaled by the mean/variance computed on just that mini-batch.
- This adds some noise to the values $z^{[l]}$ within that minibatch. So similar to dropout, it adds some noise to each hidden layer's activations.
- This has a slight regularization effect.

$$\text{mini-batch} : 64 \rightarrow 512$$

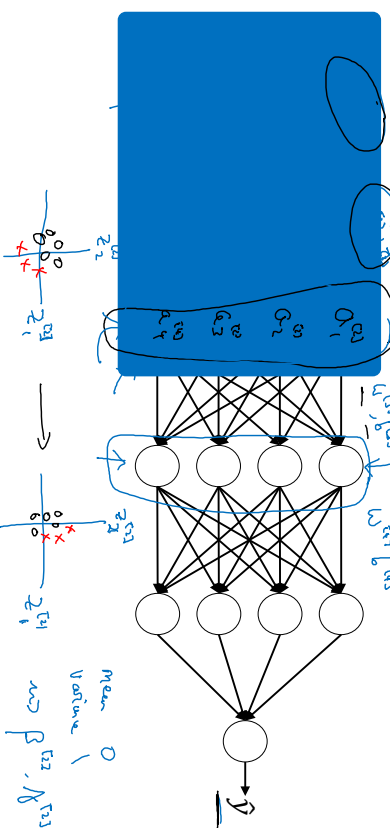
Andrew Ng

Recognizing cats, dogs, and baby chicks



Andrew Ng

Why this is a problem with neural networks?



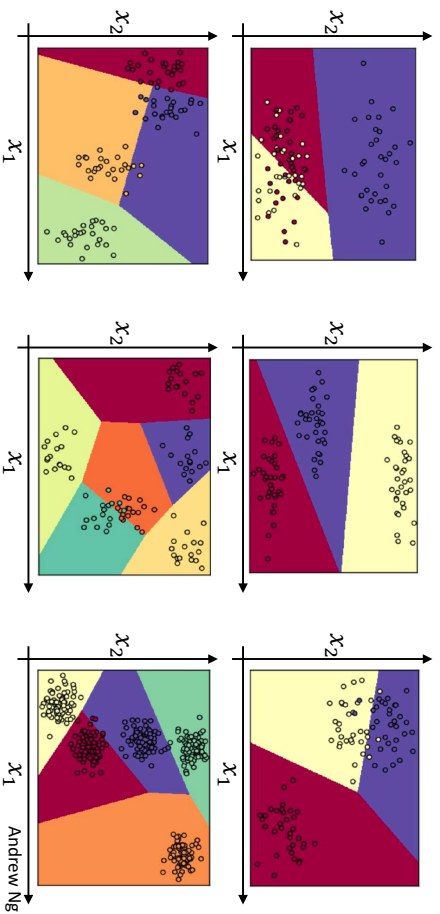
Andrew Ng



Multi-class
classification

Softmax regression

Softmax examples



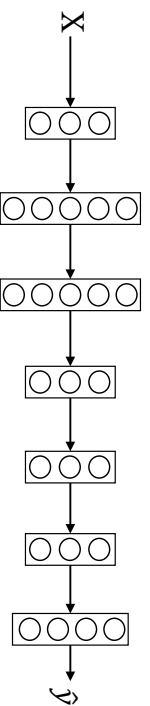
Andrew Ng

Deep learning frameworks

- Caffe/Caffe2
 - CNTK
 - DL4J
 - Keras
 - Lasagne
 - mxnet
 - PaddlePaddle
 - TensorFlow
 - Theano
 - Torch
- Choosing deep learning frameworks
- Ease of programming (development and deployment)
 - Running speed
 - - Truly open (open source with good governance)

Andrew Ng

Softmax layer



Andrew Ng



deeplearning.ai

Programming
Frameworks

Deep Learning
frameworks

Motivating problem

$$\begin{aligned} J(w) &= \boxed{w^2 - 10w + 25} \\ &\quad \uparrow \\ &\quad L(w-s)^2 \\ &\quad \uparrow \\ &\quad w = 5 \end{aligned} \quad \Bigg| \quad \begin{aligned} J(u, b) \\ \uparrow \uparrow \end{aligned}$$

Andrew Ng

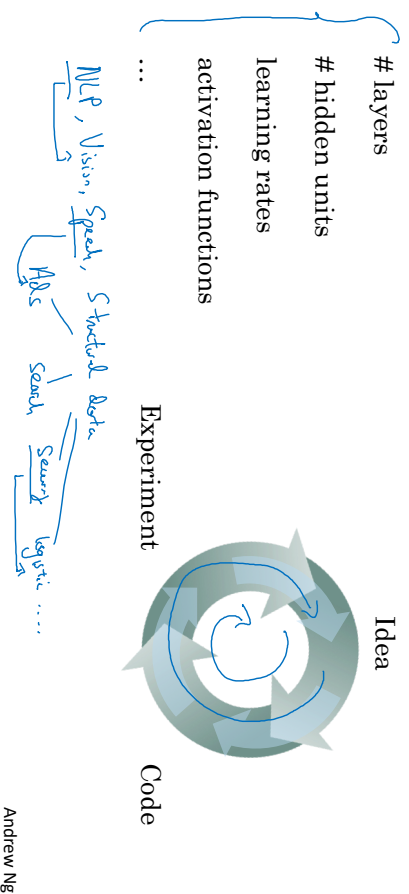
Programming
Frameworks



deeplearning.ai

TensorFlow

Applied ML is a highly iterative process



Andrew Ng



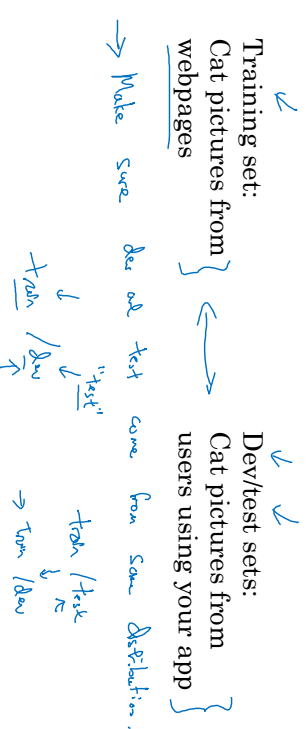
deeplearning.ai

Setting up your
ML application

Train/dev/test
sets

Mismatched train/test distribution

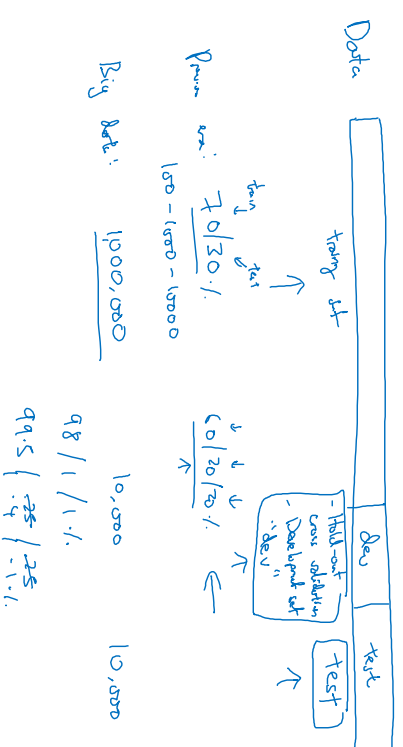
Certs



Not having a test set might be okay. (Only dev set.)

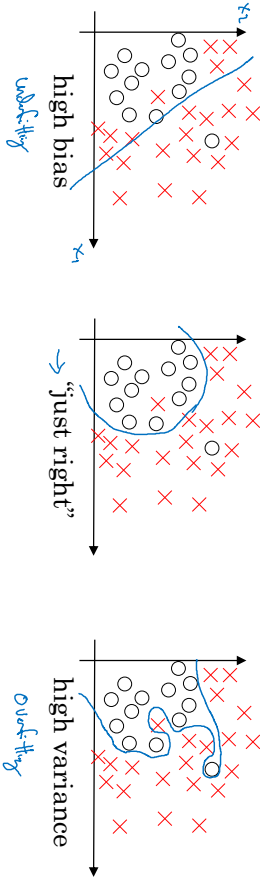
Andrew Ng

Train/dev/test sets



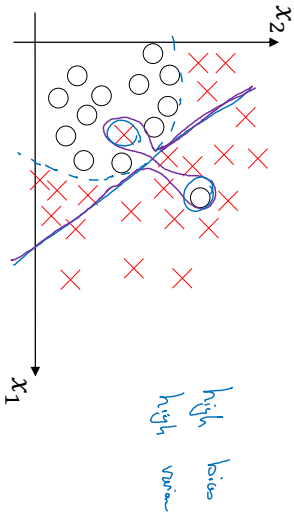
Andrew Ng

Bias and Variance



Andrew Ng

High bias and high variance



Andrew Ng

Bias and Variance

Cat classification



Train set error:	10%	15%	30%	35%
Dev set error:	110%	16%	high bias	1%
Human: 80%	high variance	high bias	high bias & high variance	low bias
Optimal (logos) error: 40%	↑	↑	↑	↑
			Blurry images	

Andrew Ng

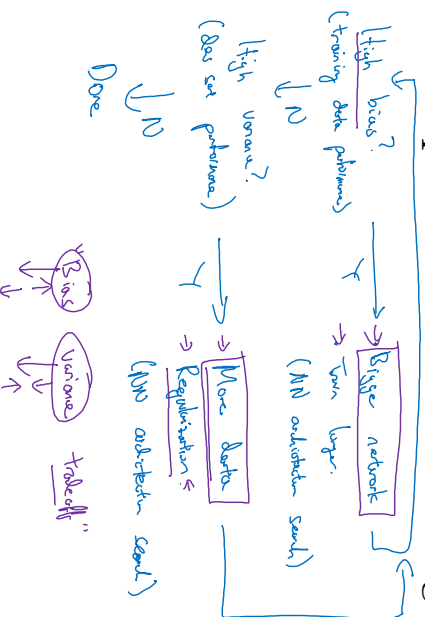


deeplearning.ai

Setting up your
ML application

Bias/Variance

Basic recipe for machine learning



Andrew Ng

Logistic regression

[illegible]

Andrew Ng

Setting up your ML application



deeplearning.ai

Basic “recipe” for machine learning

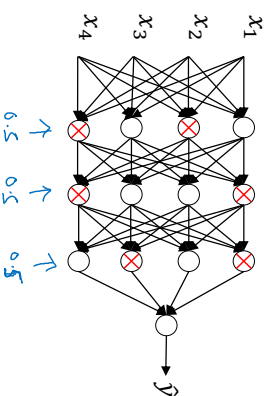
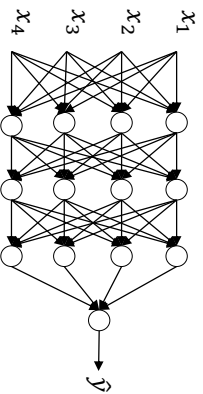
Regularizing your neural network



deeplearning.ai

Regularization

Dropout regularization



Andrew Ng

Making predictions at test time

$$a^{(2)} = X$$

$$\begin{aligned} \text{No drop out.} \\ z^{(1)} &= w^{(0)} a^{(0)} + b^{(0)} \\ a^{(1)} &= g(z^{(1)}) \\ z^{(2)} &= w^{(1)} a^{(1)} + b^{(1)} \\ a^{(2)} &= \dots \end{aligned}$$

$1 = \text{keep-prob}$

Andrew Ng

Regularizing your neural network



deeplearning.ai

Dropout regularization

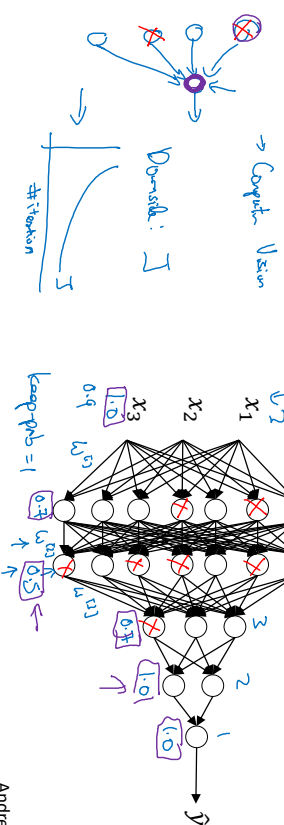
Implementing dropout (“Inverted dropout”)

$$\begin{aligned} \text{Illustrate with } \underline{L=3}. \quad \text{keep-prob} &= \underline{0.8} \\ \rightarrow \boxed{a^3} &= \frac{n \cdot \text{p.random.randn}(a^3, \text{slope}(I))}{\text{keep-prob}} < \text{keep-prob} \\ \underline{a^3} &= n \cdot \text{multiply}(a^3, \underline{I}) \quad \# a^3 * = \underline{a^3}. \\ \rightarrow \boxed{a^3 / = \text{keep-prob}} &\leftarrow \\ \text{50 units.} &\rightarrow \text{10 units shut off} \\ z^{(4)} &= w^{(3)} \cdot a^{(3)} + b^{(4)} \\ \text{rescaled by } \underline{20\%}. &\quad \underline{I_{a^3}} \\ \underline{1} &= \underline{0.8} \end{aligned}$$

Andrew Ng

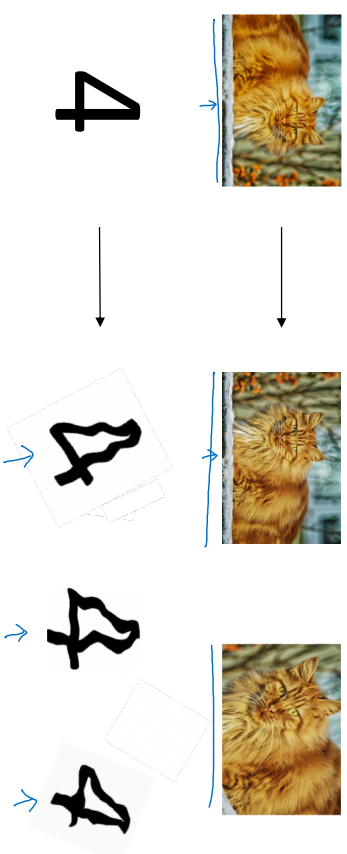
Why does drop-out work?

Intuition: Can't rely on any one feature, so have to spread out weights. $\rightarrow$ Shrink weights.



Andrew Ng

Data augmentation



Andrew Ng



deeplearning.ai

Regularizing your
neural network

Understanding
dropout



deeplearning.ai

Regularizing your
neural network

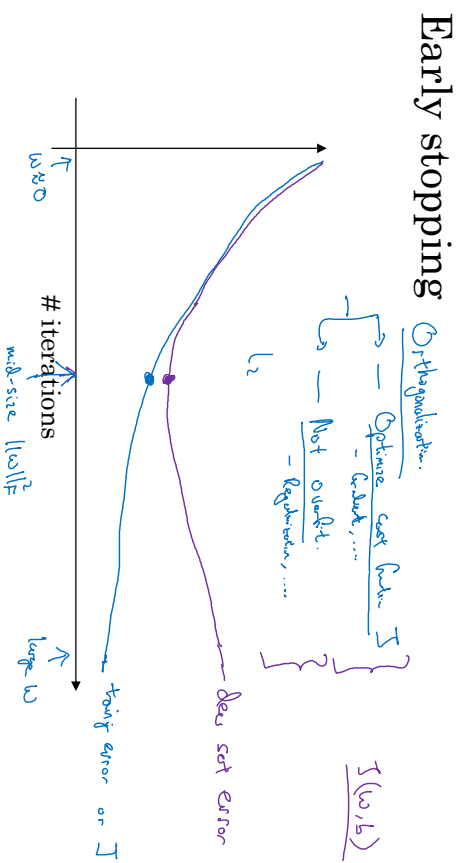
Other regularization
methods



deeplearning.ai

Setting up your optimization problem

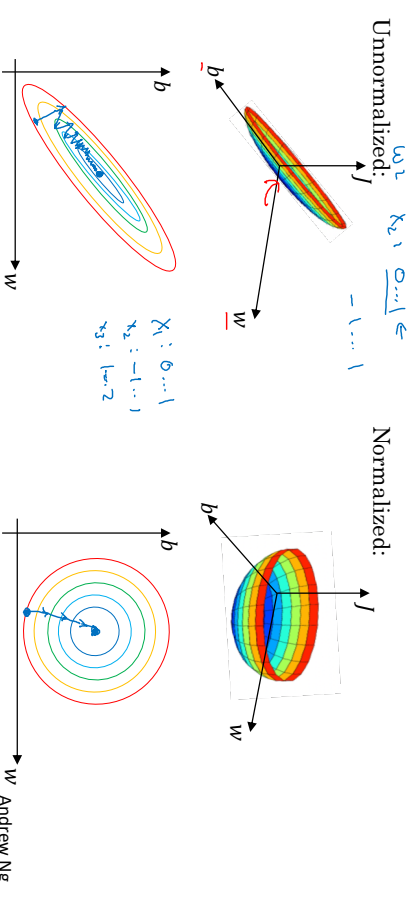
Normalizing inputs



Andrew Ng

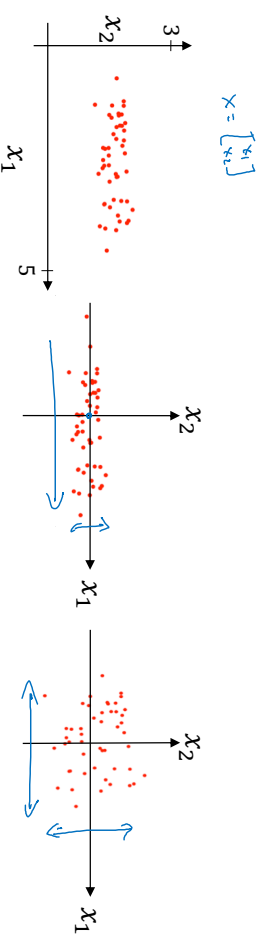
Why normalize inputs?

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$



Andrew Ng

Normalizing training sets



Subtract mean:

$$\mu = \frac{1}{n} \sum_{i=1}^n x^{(i)}$$

Use $x - \mu$

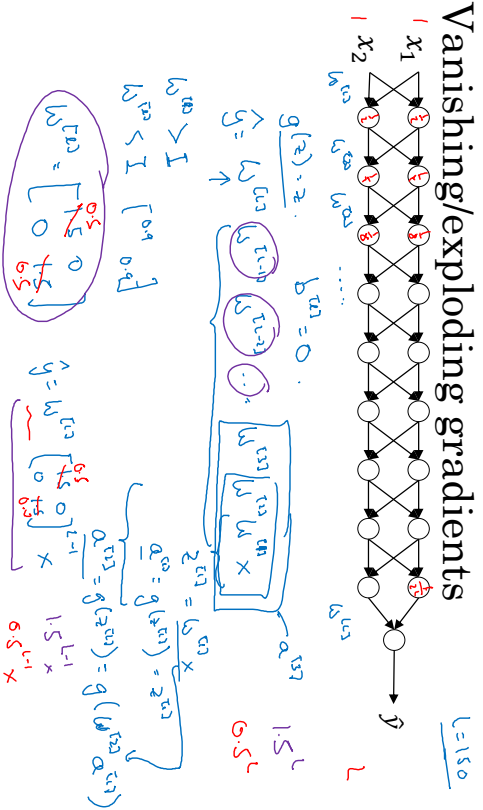
Normalize variance:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x^{(i)} - \mu)^2$$

Use $\frac{x - \mu}{\sigma}$ to normalize test set.

Andrew Ng

Vanishing/exploding gradients



Andrew Ng



deeplearning.ai

Setting up your optimization problem

Vanishing/exploding gradients

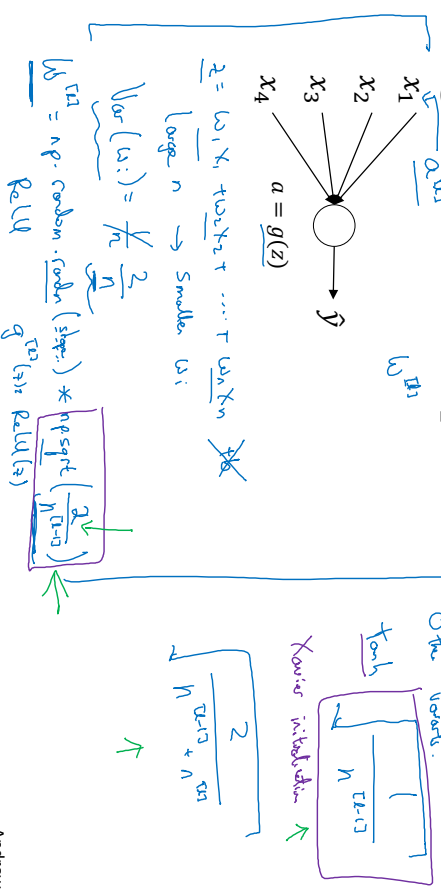


deeplearning.ai

Setting up your optimization problem

Numerical approximation of gradients

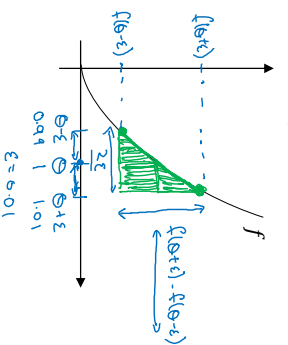
Single neuron example



Andrew Ng

Checking your derivative computation

$$f(\theta) = \theta^3$$



$$\left[\frac{f(\theta+\epsilon) - f(\theta-\epsilon)}{2\epsilon} \right] \approx g(\theta)$$

$$\frac{(1.01)^3 - (0.99)^3}{2(0.01)} = 3.0001 \approx 3$$

$$g(\theta) = 3\theta^2 = 3$$

approx error: 0.0001
prev slide: 3.0301, error: 0.03

$$f'(\theta) = \lim_{\epsilon \rightarrow 0} \frac{f(\theta+\epsilon) - f(\theta-\epsilon)}{2\epsilon} = \frac{0(\epsilon^2)}{0.01} \left| \frac{f(\theta+\epsilon) - f(\theta)}{\epsilon} \right| \text{ error: } 0(\epsilon)$$

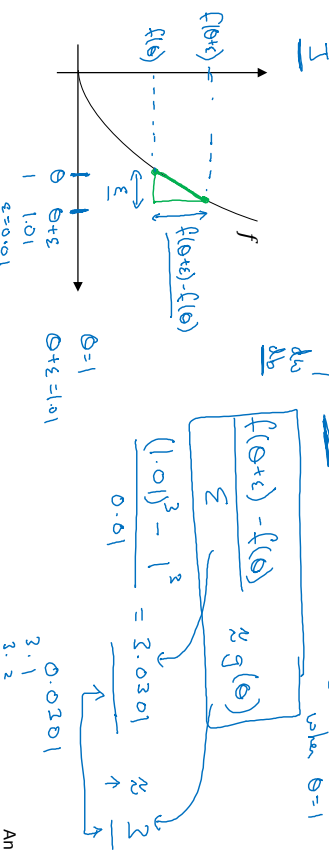
Andrew Ng

Checking your derivative computation

$$f(\theta) = \theta^3$$

$$g(\theta) = \frac{d}{d\theta} f(\theta) = f'(\theta)$$

$$g(\theta) = 3 \cdot (1)^2 = 3$$



Andrew Ng

Gradient check for a neural network

Take $\boxed{W^{[1]}}$, $\boxed{b^{[1]}}$, ..., $\boxed{W^{[L]}}$, $\boxed{b^{[L]}}$ and reshape into a big vector θ .

$$\mathcal{J}(w^{(1)}, b^{(1)}, \dots, w^{(L)}, b^{(L)}) = \mathcal{J}(\theta)$$

Take $\boxed{dW^{[1]}}$, $\boxed{db^{[1]}}$, ..., $\boxed{dW^{[L]}}$, $\boxed{db^{[L]}}$ and reshape into a big vector $d\theta$.

Is $d\theta$ the gradient of $\mathcal{J}(\theta)$?

Andrew Ng



deeplearning.ai

Setting up your
optimization problem

Gradient Checking



deeplearning.ai

Setting up your optimization problem

Gradient Checking implementation notes

Gradient checking (Grad check)

for each i :

$$\rightarrow \frac{\partial \Theta_{\text{approx}}[i]}{\partial \Theta_i} = \frac{J(\Theta_1, \Theta_2, \dots, \Theta_i + \epsilon, \dots) - J(\Theta_1, \Theta_2, \dots, \Theta_i - \epsilon, \dots)}{2\epsilon}$$

$J(\Theta) = J(\Theta_1, \Theta_2, \dots)$

$$\approx \frac{\partial J}{\partial \Theta_i} \quad \left| \quad \frac{\partial \Theta_{\text{approx}}}{\partial \Theta_i} \approx \frac{\partial J}{\partial \Theta_i} \right.$$

Check

$$\frac{\|\partial \Theta_{\text{approx}} - \partial J\|_2}{\|\partial \Theta_{\text{approx}}\|_2 + \|\partial J\|_2} \approx \frac{\left[\begin{array}{c} 10^{-7} \\ 10^{-5} \end{array} - \text{grad!} \right]}{10^{-5}} \leftarrow$$

$$\rightarrow 10^{-3} - \text{wrong!} \leftarrow$$

$$\epsilon = 10^{-7}$$

Andrew Ng

Gradient checking implementation notes

- Don't use in training – only to debug $\frac{\partial \Theta_{\text{approx}}[i]}{\partial \Theta_i} \leftrightarrow \frac{\partial J}{\partial \Theta_i}$
- If algorithm fails grad check, look at components to try to identify bug.
 - $\frac{\partial b}{\partial \Theta} = \frac{\partial w}{\partial \Theta}$
- Remember regularization.

$$J(\Theta) = \frac{1}{n} \sum \frac{1}{2} (y^{(i)} - \hat{y}^{(i)})^2 + \frac{\lambda}{2n} \sum \|\omega^{(i)}\|_2^2$$

$$\partial \Theta = \text{grad} \text{ of } J \text{ w.r.t. } \Theta$$
- Doesn't work with dropout. J $\frac{\partial \text{keep-prob} = 1.0}{\partial \Theta}$
- Run at random initialization; perhaps again after some training.

LS, b, w, 0

Andrew Ng

Batch vs. mini-batch gradient descent

Vectorization allows you to efficiently compute on m examples.

$$\begin{aligned}
 X &= \begin{bmatrix} x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ x^{(1)}_1 & x^{(2)}_1 & \dots & x^{(m)}_1 \\ \vdots & \vdots & \ddots & \vdots \\ x^{(1)}_n & x^{(2)}_n & \dots & x^{(m)}_n \end{bmatrix} \\
 y &= \begin{bmatrix} y^{(1)} & y^{(2)} & \dots & y^{(m)} \\ y^{(1)}_1 & y^{(2)}_1 & \dots & y^{(m)}_1 \\ \vdots & \vdots & \ddots & \vdots \\ y^{(1)}_n & y^{(2)}_n & \dots & y^{(m)}_n \end{bmatrix} \\
 \theta &= \begin{bmatrix} \theta_0 & \theta_1 \\ \theta_0 & \theta_1 \\ \vdots & \vdots \\ \theta_0 & \theta_1 \end{bmatrix} \\
 \text{if } m = 5000, \text{ then } & \text{mini-batches of } 1000 \text{ each} \\
 \text{mini-batches } t: & X^{t+1}, y^{t+1}
 \end{aligned}$$

Andrew Ng



deeplearning.ai

Optimization Algorithms

Mini-batch

gradient descent



deeplearning.ai

Optimization Algorithms

Understanding mini-batch gradient descent

Mini-batch gradient descent

for $t = 1, \dots, 5000$ {

Formal prep on X^{t+1} .

$$X^{t+1} = W^{(0)} X^{t+1} + b^{(0)}$$

Vectorized implementation (1000 examples)

$$A^{(t+1)} = g^{(t+1)}(Z^{(t+1)})$$

$$\text{Compute cost } J^{(t+1)} = \frac{1}{1000} \sum_{i=1}^{1000} d(y^{(i)}, \hat{y}^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_{i=1}^{1000} \|W^{(i+1)}\|_F^2$$

Backprop to compute gradients w.r.t $J^{(t+1)}$ (using $y^{(i+1)}, y^{(i+1)}$)

$$W^{(t+1)} = W^{(t)} - \alpha dW^{(t)}, \quad b^{(t+1)} = b^{(t)} - \alpha db^{(t)}$$

"1 epoch" pass through training set.

1 step of gradient descent using X^{t+1}, y^{t+1} (as if $m=1000$)

$$X, y$$

Andrew Ng

Choosing your mini-batch size

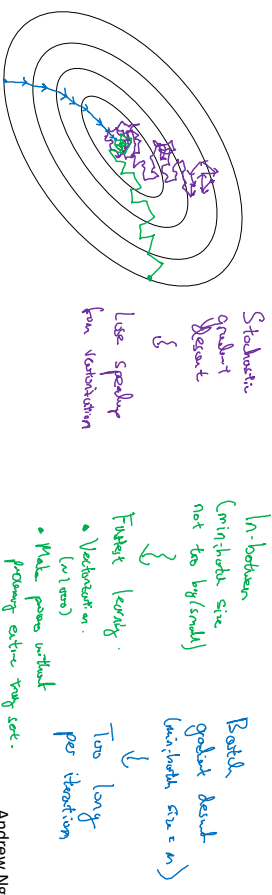
→ If mini-batch size = m : Batch gradient descent.

$$(X^{(1)}, Y^{(1)}) = (X, Y).$$

→ If mini-batch size = 1 : Stochastic gradient descent. Every example is its own mini-batch.

$$(X^{(1)}, Y^{(1)}) = (x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$$

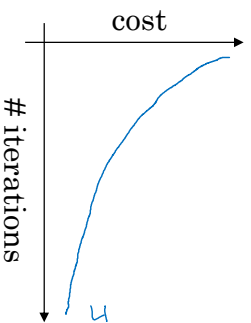
In practice: Several mini-batches $\frac{1}{m}$ and m



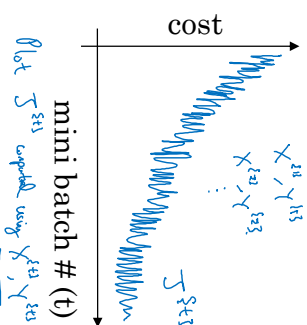
Andrew Ng

Training with mini batch gradient descent

Batch gradient descent



Mini-batch gradient descent



Plot $J^{(32)}$ computed using $X^{(32)}, Y^{(32)}$

Andrew Ng



deeplearning.ai

Optimization
Algorithms

Exponentially
weighted averages

Choosing your mini-batch size

If small toy set : Use batch gradient descent.
($m \leq 2000$)

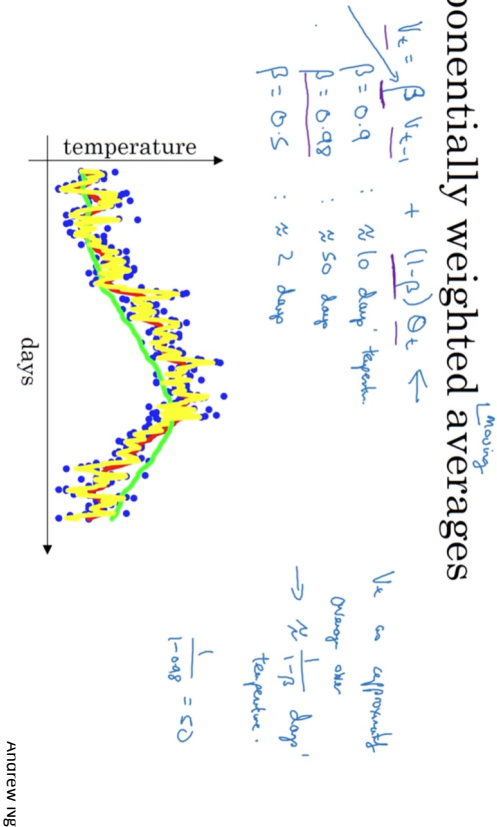
Typical mini-batch sizes:

→ 64, 128, 256, 512
2⁶ 2⁷ 2⁸ 2⁹

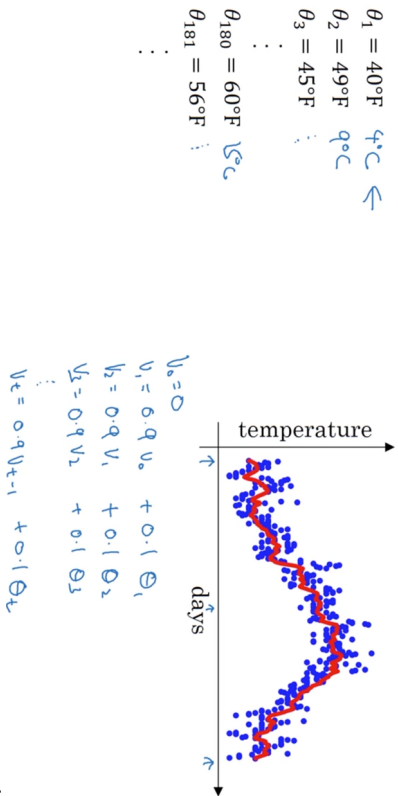
Make sure mini-batch fit in CPU/GPU memory.
 $X^{(143)}, Y^{(148)}$

Andrew Ng

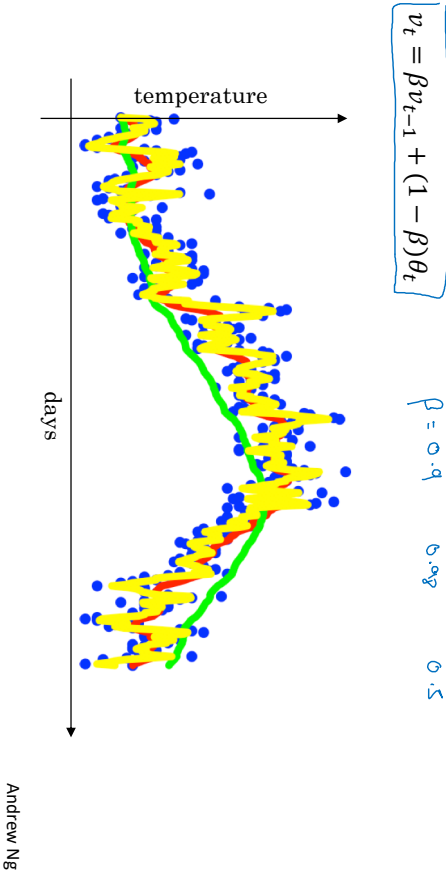
Exponentially weighted averages



Temperature in London



Exponentially weighted averages



deeplearning.ai

Optimization
Algorithms

Understanding
exponentially
weighted averages

Implementing exponentially weighted averages

$$\begin{aligned} v_0 &= 0 \\ v_1 &= \beta v_0 + (1 - \beta) \theta_1 \\ v_2 &= \beta v_1 + (1 - \beta) \theta_2 \\ v_3 &= \beta v_2 + (1 - \beta) \theta_3 \\ &\dots \end{aligned}$$

$$\begin{aligned} v_\theta &:= 0 \\ v_\theta &:= \beta v + (1 - \beta) \theta_1 \\ v_\theta &:= \beta v + (1 - \beta) \theta_2 \\ &\vdots \\ &\rightarrow v_\theta = 0 \\ \text{Keep } \{ & \\ \text{Get next } \theta_t & \\ v_\theta &:= \beta v_\theta + (1 - \beta) \theta_t \leftarrow \end{aligned}$$

Andrew Ng

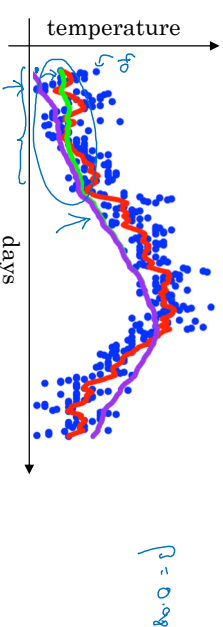
Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1 - \beta) \theta_t$$

$$\begin{aligned} v_{100} &= 0.9v_{99} + 0.1\theta_{100} \\ v_{99} &= 0.9v_{98} + 0.1\theta_{99} \\ v_{98} &= 0.9v_{97} + 0.1\theta_{98} \\ &\dots \\ v_{100} &= 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9(0.1\theta_{98} + 0.9(\dots))) \\ &= 0.1\theta_{100} + 0.1 \times 0.9 \cdot \theta_{99} + 0.1(0.9)^2 \theta_{98} + 0.1(0.9)^3 \theta_{97} + \dots \\ &\quad 0.1 \times 0.9^0 \theta_{100} \approx \frac{1}{e} \quad \frac{(1-\epsilon)^k}{0.9} \leftarrow \frac{1}{e} \quad 0.98^k? \\ &\quad \epsilon = 0.02 \rightarrow 0.98^{50} \approx \frac{1}{e} \end{aligned}$$

Andrew Ng

Bias correction



$$\begin{aligned} &\rightarrow v_t = \beta v_{t-1} + (1 - \beta) \theta_t \\ v_0 &= 0 \\ v_1 &= 0.98v_0 + 0.02\theta_1 \\ v_2 &= 0.98v_1 + 0.02\theta_2 \\ &= 0.98 \times 0.02 \times \theta_1 + 0.02\theta_2 \\ &= 0.0196\theta_1 + 0.02\theta_2 \end{aligned}$$

Andrew Ng

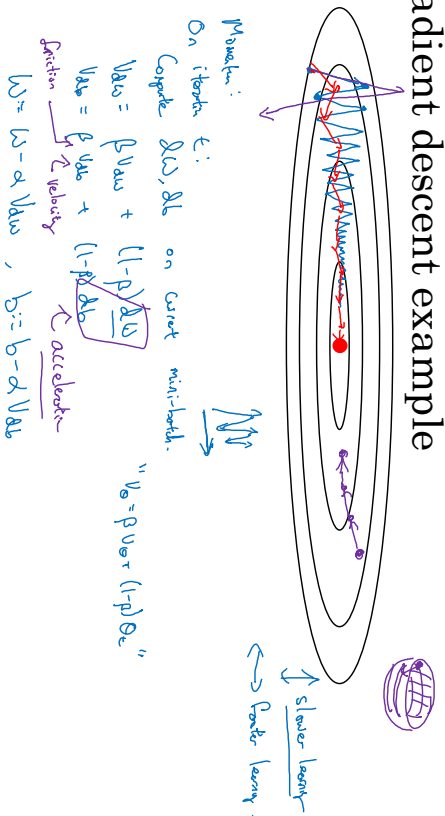


deeplearning.ai

Optimization Algorithms

Bias correction in exponentially weighted average

Gradient descent example



Andrew Ng



deeplearning.ai

Optimization Algorithms

Gradient descent with momentum



deeplearning.ai

Optimization Algorithms

RMSprop

Implementation details

$$V_{AB} = 0, V_{AC} = 0$$

On iteration t :

Compute dW, db on the current mini-batch

$$\begin{aligned} & \rightarrow v_{dW} = \beta v_{dW} + \underbrace{(\cancel{1 - \beta})}_{\text{Max}} dW \\ & \rightarrow v_{ab} = \beta v_{ab} + \underbrace{(1 - \beta)}_{\text{Max}} db \\ & W = W - \underbrace{\alpha v_{dW}}_{\text{Max}}, \quad b = \underbrace{b - \alpha v_{db}}_{\text{Max}} \end{aligned}$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

Hyperparameters: α, β

Hyperparameters: α, β

$\beta = 0.9$
Average over last 10 gradients

Andrew Ng



deeplearning.ai

Optimization Algorithms

Adam optimization algorithm

Hyperparameters choice:

$\rightarrow \alpha$: needs to be tune
 $\rightarrow \beta_1: 0.9 \rightarrow (\underline{dw})$
 $\rightarrow \beta_2: 0.999 \rightarrow (\underline{dw}^2)$
 $\rightarrow \epsilon: 10^{-8}$

Adam: Adaptive moment estimation



Adam Coates

Andrew Ng

Adam optimization algorithm

$v_{dw} = 0, S_{dw} = 0, v_{db} = 0, S_{db} = 0$
 On iteration t :
 Compute dw, db using curved mini-batch
 $v_{dw} = \beta_1 v_{dw} + (1 - \beta_1) dw$, $v_{db} = \beta_1 v_{db} + (1 - \beta_1) db \leftarrow \text{"moment"} \beta_1$
 $S_{dw} = \beta_2 S_{dw} + (1 - \beta_2) dw^2$, $S_{db} = \beta_2 S_{db} + (1 - \beta_2) db^2 \leftarrow \text{"RMSprop"} \beta_2$
 $v_{dw}^{correct} = v_{dw} / (1 - \beta_1^t)$, $v_{db}^{correct} = v_{db} / (1 - \beta_1^t)$
 $S_{dw}^{correct} = S_{dw} / (1 - \beta_2^t)$, $S_{db}^{correct} = S_{db} / (1 - \beta_2^t)$
 $w := w - \alpha \frac{v_{dw}^{correct}}{\sqrt{S_{dw}^{correct} + \epsilon}}$, $b := b - \alpha \frac{v_{db}^{correct}}{\sqrt{S_{db}^{correct} + \epsilon}}$

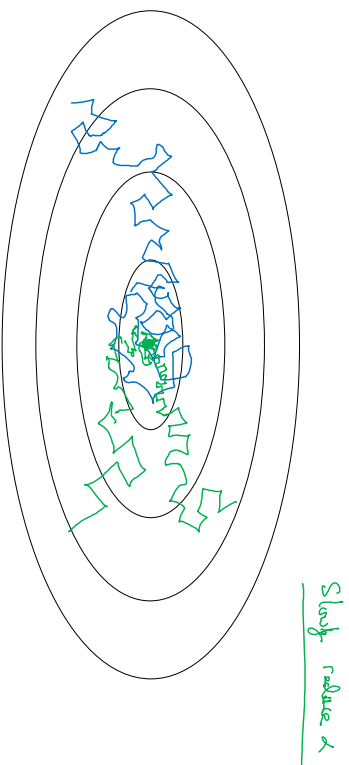
Andrew Ng

Andrew Ng

RMSprop

On iteration t :
 Compute dw, db on curved mini-batch
 $S_{dw} = \beta_1 S_{dw} + (1 - \beta_1) dw^2 \leftarrow \text{"element-wise"} \beta_1$
 $S_{db} = \beta_2 S_{db} + (1 - \beta_2) db^2 \leftarrow \text{"large"} \beta_2$
 $w := w - \alpha \frac{dw}{\sqrt{S_{dw} + \epsilon}}$, $b := b - \alpha \frac{db}{\sqrt{S_{db} + \epsilon}}$
 $\epsilon = 10^{-8}$

Learning rate decay



Andrew Ng

Other learning rate decay methods

$$\alpha = 0.95^{\text{epoch} - \text{min}} \cdot \alpha_0 \quad \text{-- exponential decay}$$

$$\alpha = \frac{k}{\sqrt{\text{epoch} - \text{min}}} \cdot \alpha_0 \quad \text{or} \quad \frac{k}{\sqrt{t}} \cdot \alpha_0$$

for t

staircase staircase

Manual decay:

A graph showing a staircase-like decay of the learning rate over time. The x-axis is labeled t and the y-axis is labeled α . The line starts at a high value and then drops sharply to a lower value, repeating this pattern multiple times, creating a staircase effect.

Andrew Ng



deeplearning.ai

Optimization Algorithms

Learning rate decay

Learning rate decay

1 epoch = 1 pass through data.

$$\alpha = \frac{1}{1 + \text{decay-rate} \times \text{epoch} - \text{min}}$$

epoch	α
1	0.1
2	0.67
3	0.5
4	0.4
$\vdots$	$\vdots$

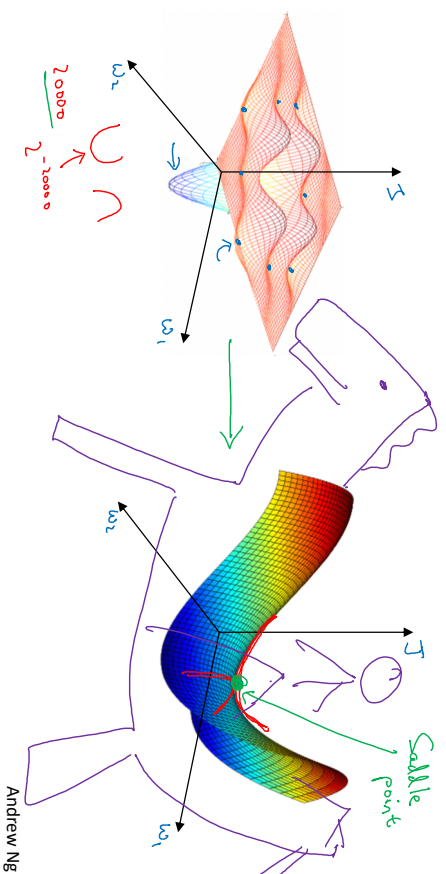
$\begin{bmatrix} x_{in} & x_{in} & \dots \end{bmatrix}$
 $\downarrow$
 epoch 1
 $\downarrow$
 epoch 2
 $\vdots$

$\alpha_0 = 0.2$
 $\text{decay-rate} = 1$



Andrew Ng

Local optima in neural networks



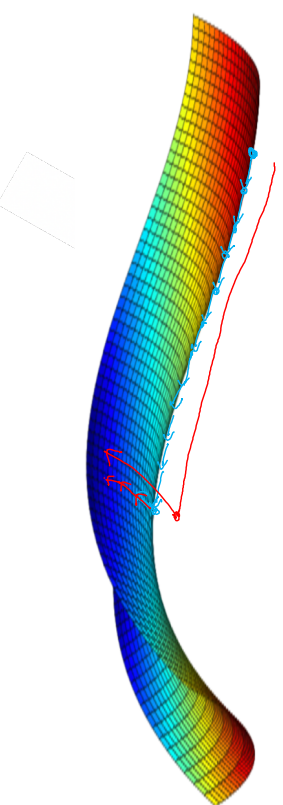
Optimization Algorithms

The problem of local optima



deeplearning.ai

Problem of plateaus



- Unlikely to get stuck in a bad local optima
- Plateaus can make learning slow